WEEK 6 TASK

Web Security & VAPT Practical Report

DVWA • SQL Injection • XSS • Burp Suite • OWASP ZAP

Name	Prajapati Meet Ghanshyam
Organization	DG Interns Hub
Date	28 October 2026
Environment	Kali Linux, Docker, local DVWA at http://localhost:4280

Scope note: Evidence in this report comes from the screenshots provided for the local, intentionally vulnerable DVWA training application. The results are not claims about an external production website.

1. Executive Summary

For Week 6, a local DVWA application was launched with Docker and tested to learn common web application vulnerabilities. The screenshots show SQL Injection behaviour, Reflected XSS and Stored XSS demonstrations, HTTP traffic inspection in Burp Suite, and an OWASP ZAP automated scan.

Key outcomes

• SQL Injection: a crafted input returned multiple records where normal input returned a single record.

• Reflected XSS: the browser displayed an alert labelled “XSS Test”.

• Stored XSS: a guestbook payload triggered an alert labelled “stored XSS Test”.

• Burp Suite: HTTP history captured local requests and responses, including HTTP 200 OK and 302 Found.

• OWASP ZAP: the scan completed; console output reported 87 alerts and the Alerts pane showed 12 alert groups.

Interpretation: DVWA is intentionally vulnerable. ZAP alerts require individual review and are not automatically proof that every alert is exploitable.

2. Objectives, Scope and Method

Objectives

• Understand SQL Injection and cross-site scripting (XSS).

• Capture browser HTTP traffic and inspect request/response headers.

• Run an automated scan with OWASP ZAP and record observations.

• Document evidence, risk context and remediation recommendations.

Scope

Target: http://localhost:4280 (local DVWA instance). Security level was set to Low for the training demonstration. Screenshots show Kali Linux, Docker, the DVWA browser, Burp Suite Community Edition and OWASP ZAP 2.17.0.

Method

1. Start DVWA using Docker Compose. 2. Open the local application and configure the lab. 3. Submit controlled SQLi and XSS test inputs. 4. Inspect requests and responses in Burp Suite. 5. Run ZAP Automated Scan. 6. Save screenshots and document findings.

Ethical note: Keep intentionally vulnerable applications isolated and local. Only test systems for which explicit authorization has been granted.

3. Lab Setup

Docker Compose output shows DVWA-related images pulled and containers started. The browser then displayed the DVWA database setup page and later the application security page.

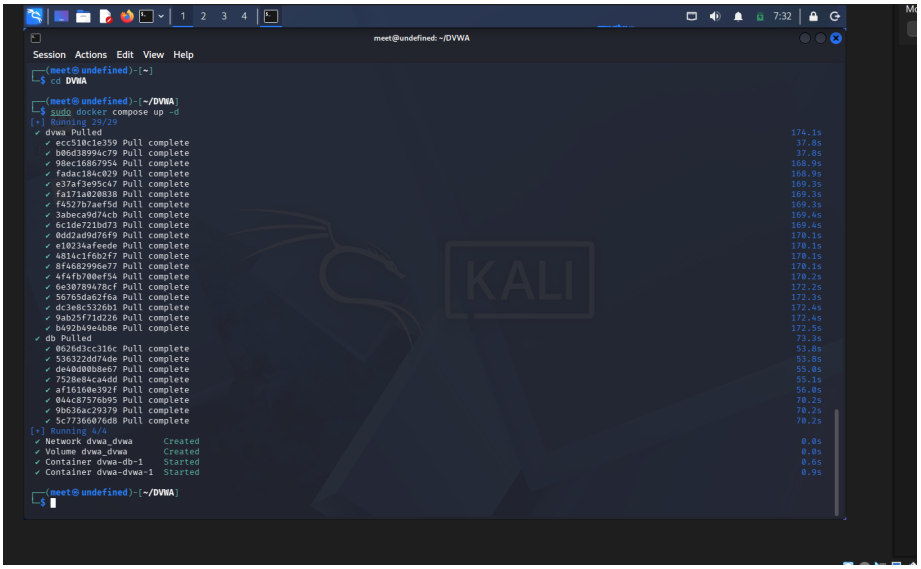


Figure 1. Docker Compose output showing containers started.

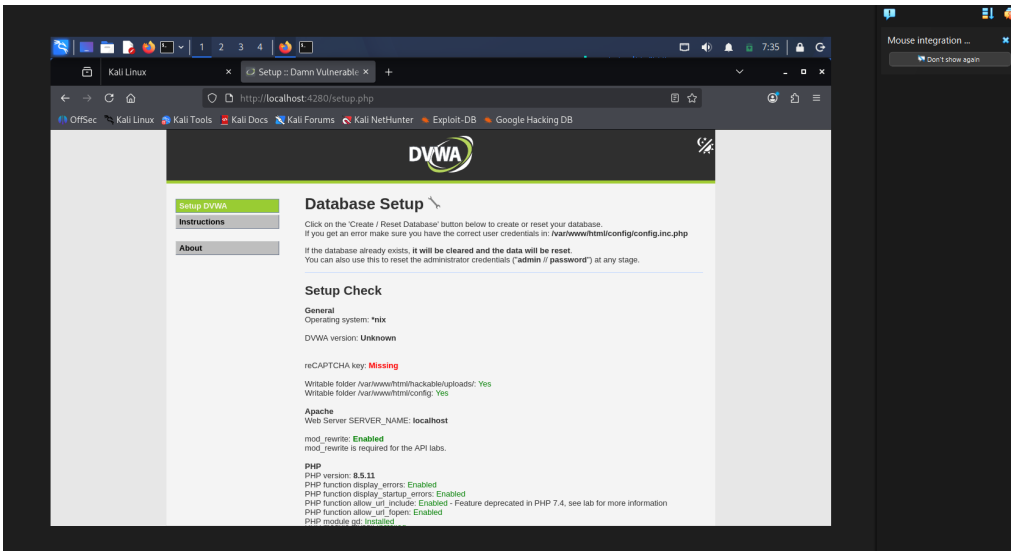


Figure 2. DVWA database setup page.

4. SQL Injection

The security level was set to Low for the controlled DVWA exercise. A normal input of 1 returned the admin record. The crafted input visible in the supplied screenshot returned multiple records, which is evidence of SQL Injection behaviour in this deliberately vulnerable lab.

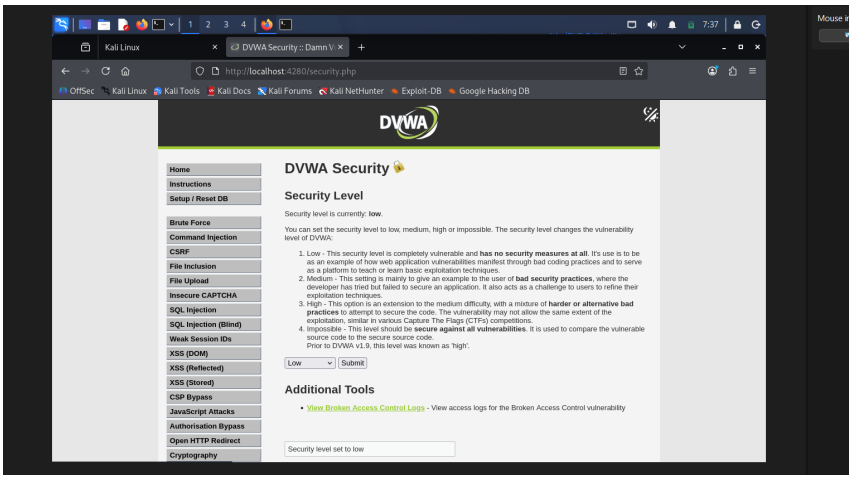


Figure 3. DVWA security level set to Low.

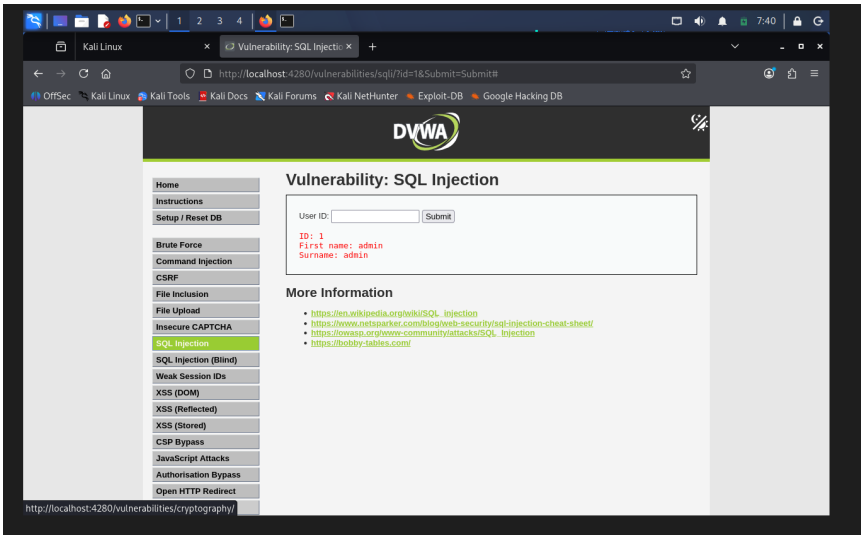


Figure 4. Normal input returning a single record.

4.1 SQL Injection — comparison

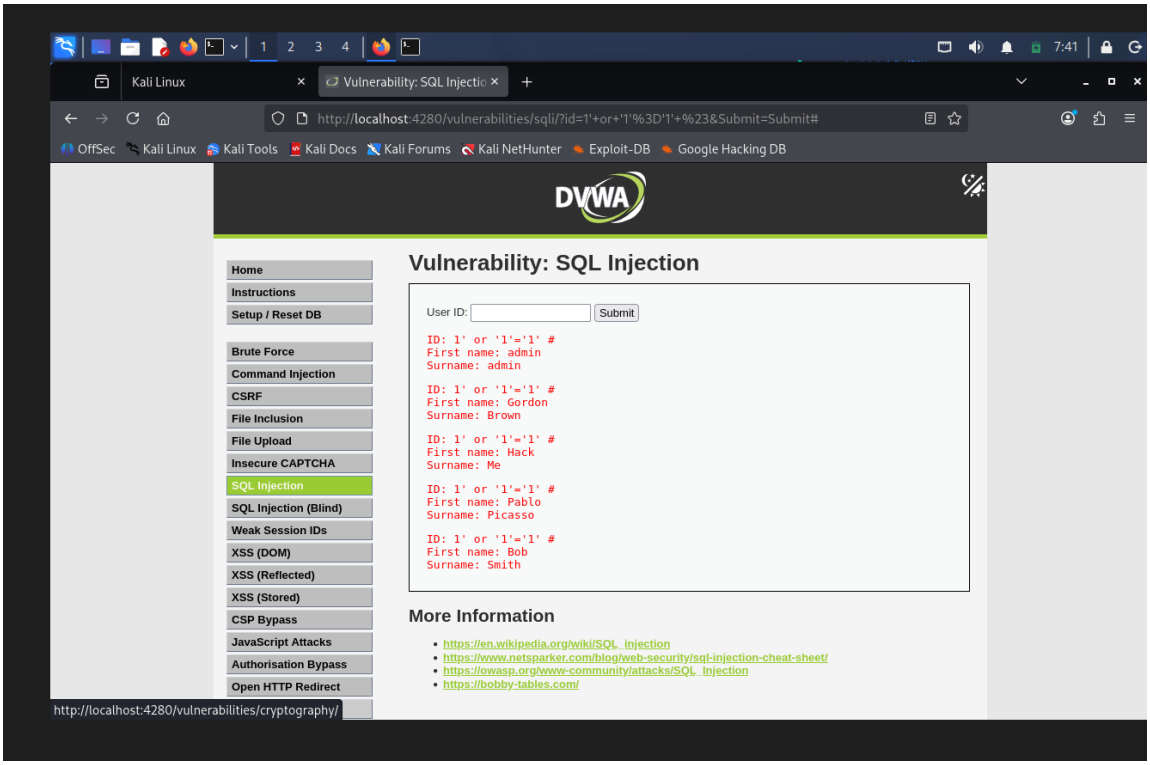


Figure 5. Crafted input returning multiple records in DVWA.

Finding 1 — SQL Injection behaviour observed. Risk in the intentionally vulnerable lab is high; real-world severity depends on the data, privileges, exposure and exploitability. Recommended fix: parameterized queries/prepared statements, server-side validation and least-privilege database access.

5. Cross-Site Scripting (XSS)

Reflected XSS: the browser displayed an alert with “XSS Test” after a test payload was submitted. This shows script execution from reflected input in the local training target.

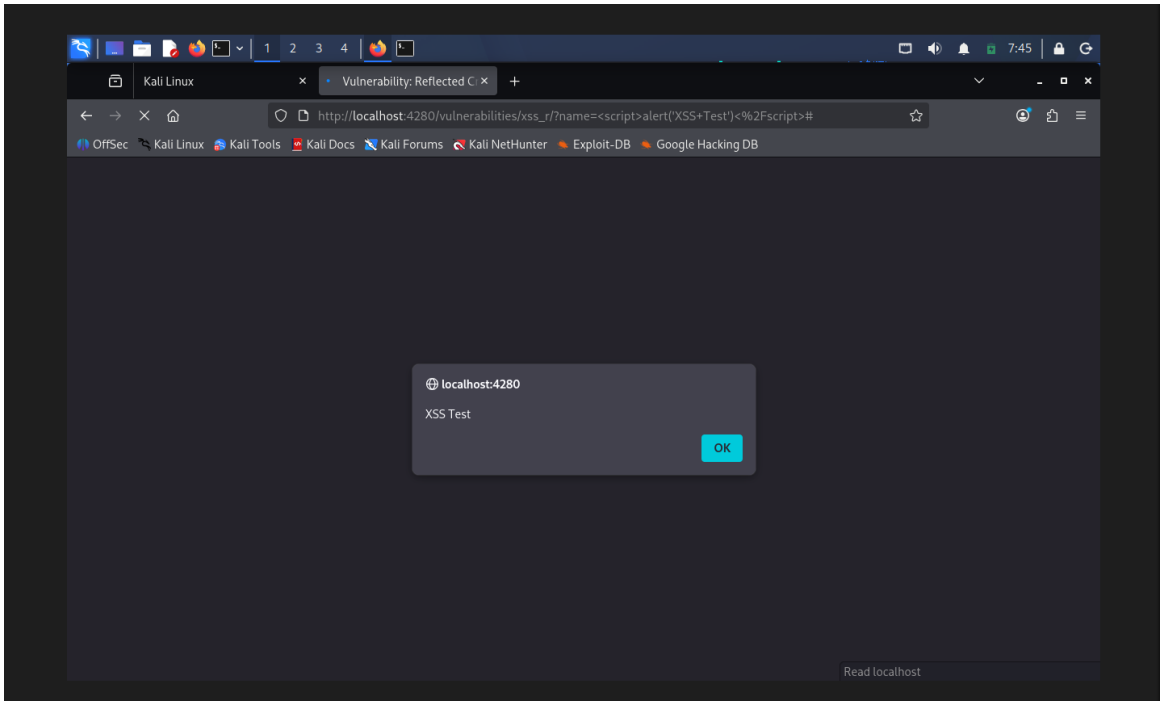


Figure 6. Reflected XSS alert shown in the browser.

Finding 2 — Reflected XSS demonstrated in DVWA. Mitigation: context-aware output encoding, safe templating and review of unsafe DOM sinks. A Content Security Policy is useful as defence in depth.

5.1 Stored XSS

A guestbook entry was submitted using the name TestUser and a script test payload. The browser displayed an alert reading “stored XSS Test”, showing stored XSS behaviour in the local DVWA application.

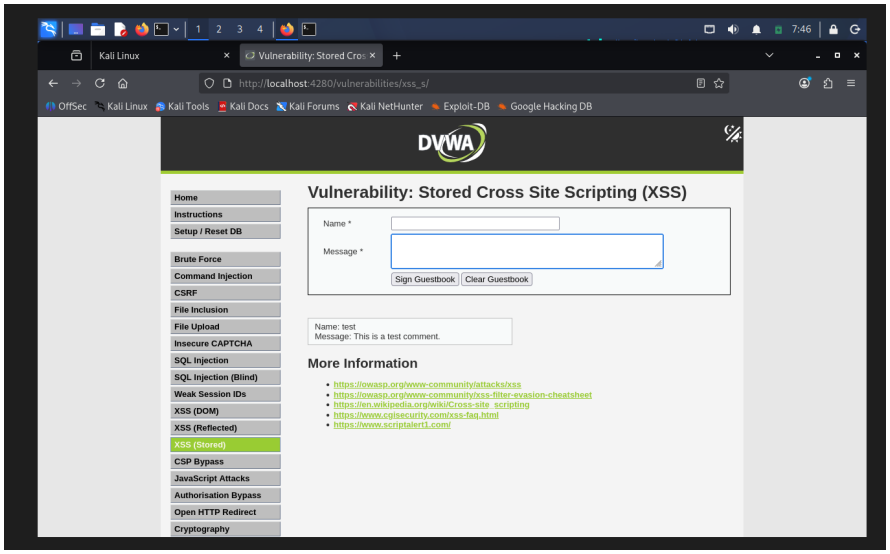


Figure 7. Stored XSS guestbook form.

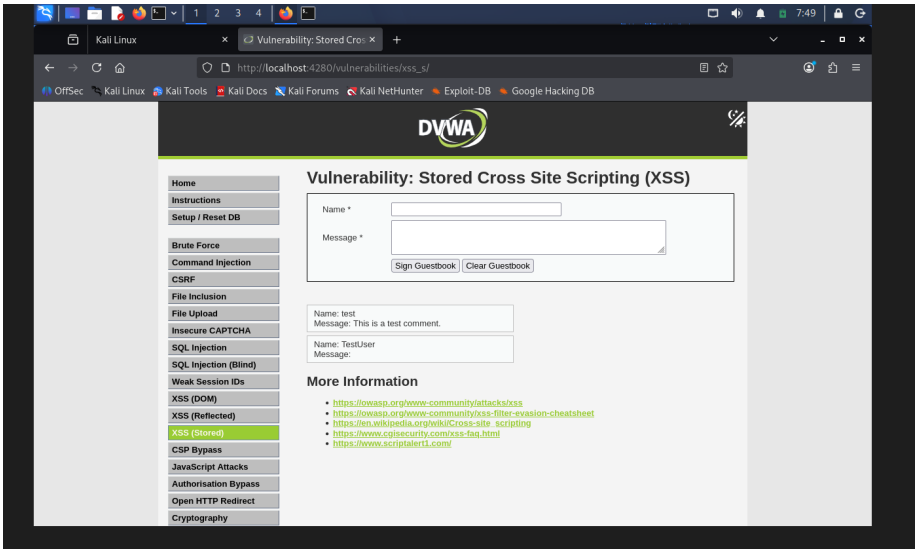


Figure 8. Stored XSS test alert.

Finding 3 — Stored XSS demonstrated in DVWA. Mitigation: encode output at render time, sanitize permitted rich text with a maintained library and avoid unsafe HTML insertion.

6. Burp Suite HTTP Inspection

Burp Suite Community Edition was used to review browser traffic to localhost:4280. HTTP history included requests such as GET /, GET /login.php, POST /login.php and requests to the SQL Injection page. Captured responses included 200 OK and 302 Found.

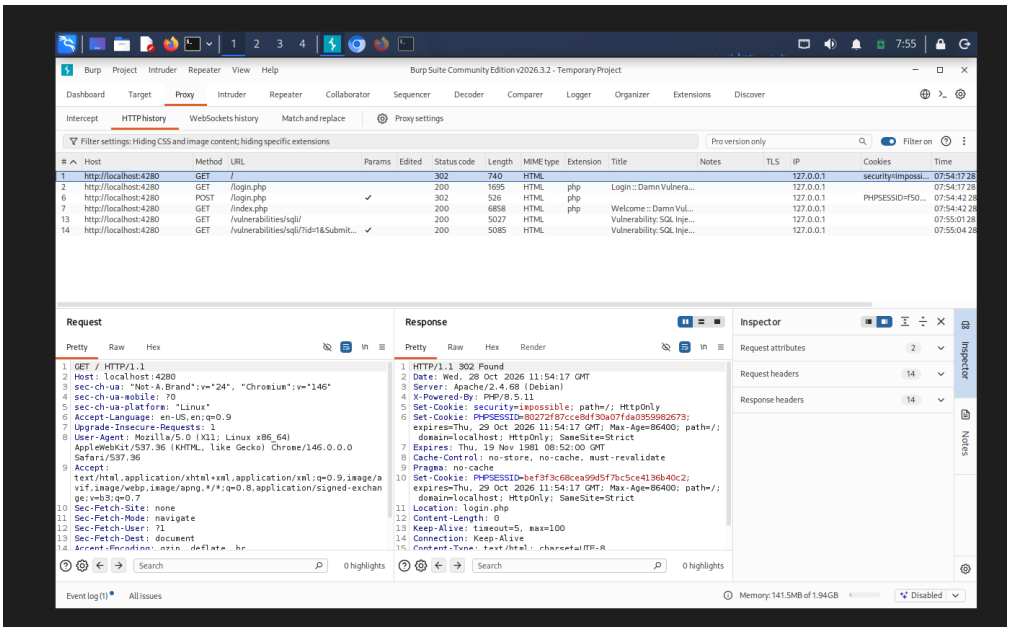


Figure 9. Burp Suite HTTP history showing local DVWA requests.

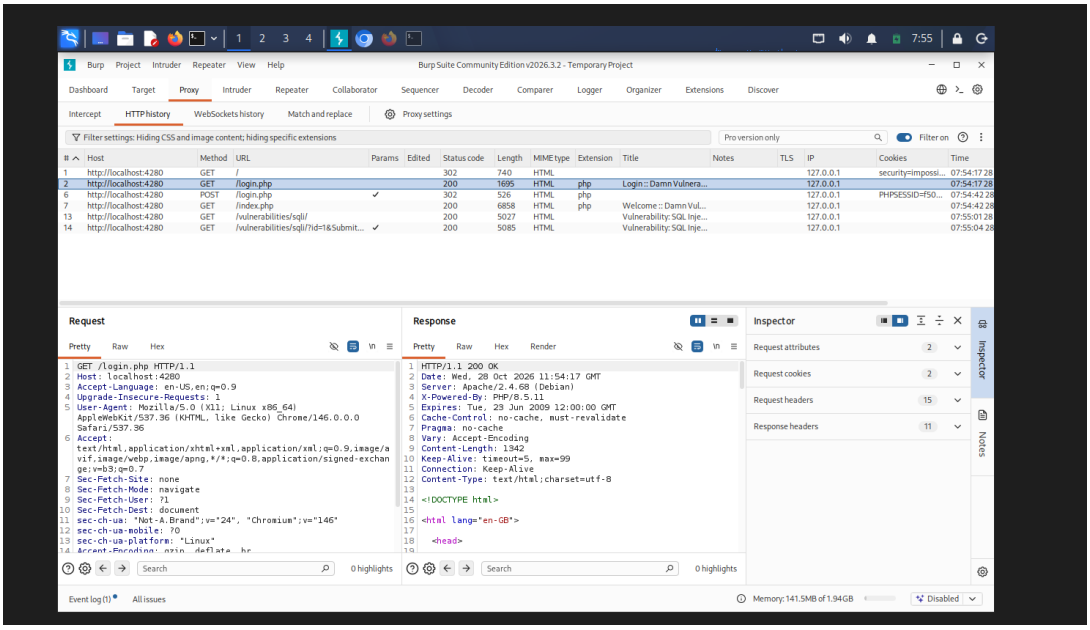


Figure 10. Request and response panels with HTTP status and headers.

Privacy reminder: session cookies and identifiers may be visible in headers. Redact these values before publishing screenshots or reports publicly.

7. OWASP ZAP Automated Scan

OWASP ZAP 2.17.0 Automated Scan was run against http://localhost:4280. The console reported that the scan completed with 87 alerts raised; the Alerts pane in the screenshot displayed 12 alert groups. These are scanner-generated observations and must be validated individually.

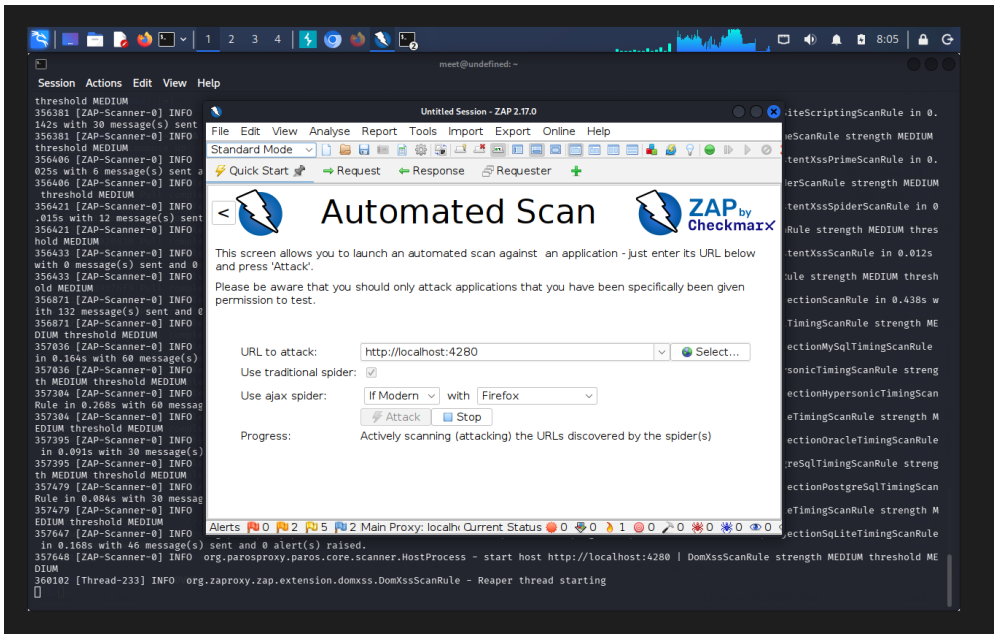


Figure 11. ZAP Automated Scan screen targeting the local DVWA instance.

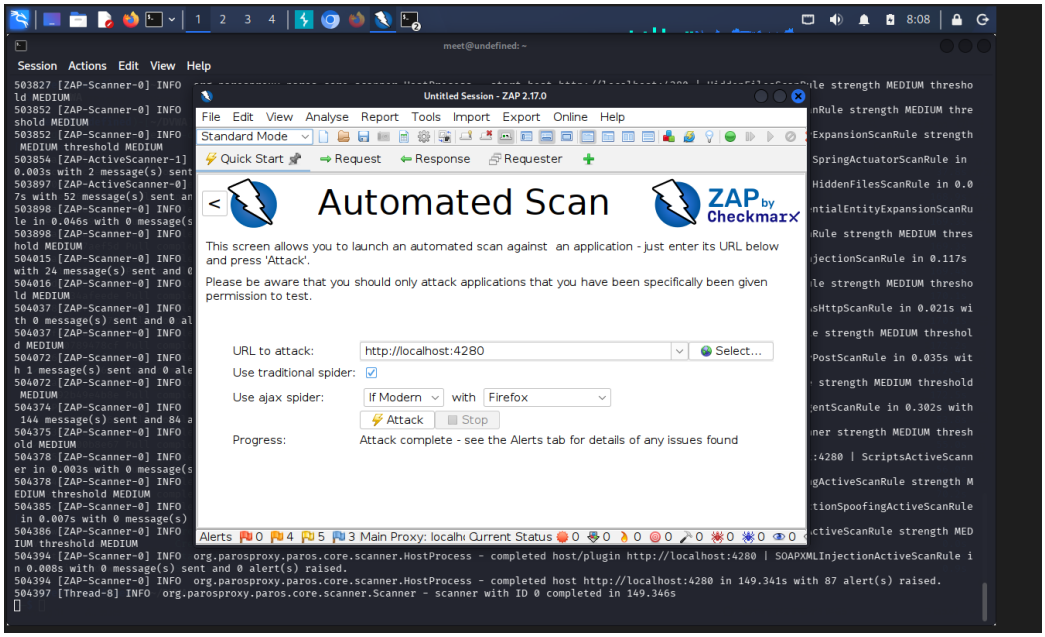


Figure 12. ZAP reports the scan completed.

8. ZAP Alerts and Risk Interpretation

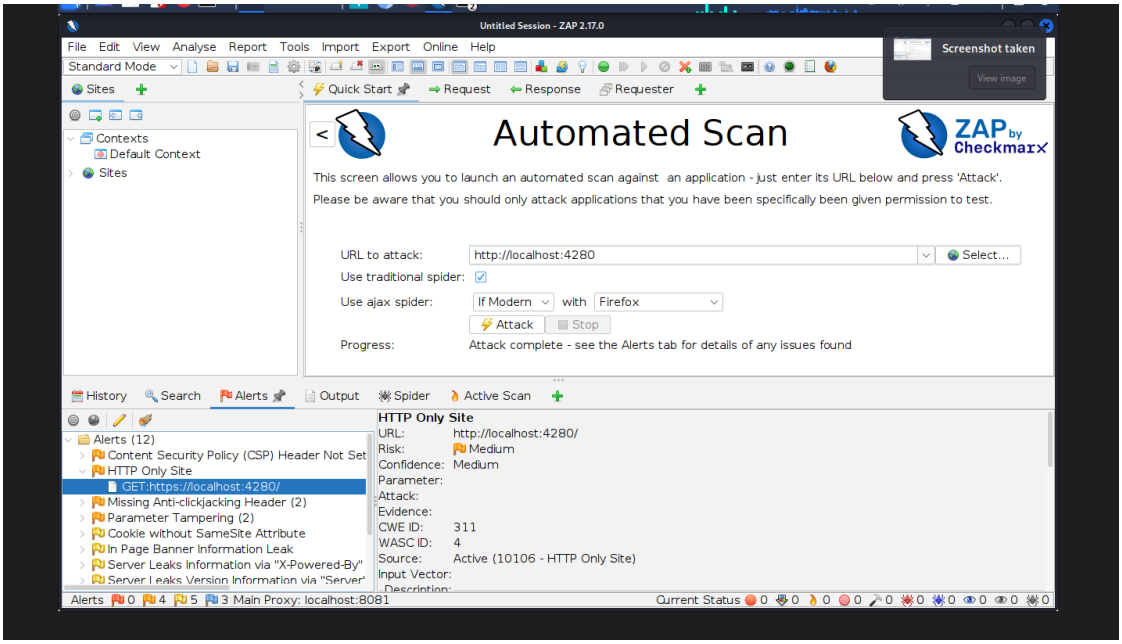


Figure 13. ZAP Alerts pane with a selected HTTP Only Site finding.

The visible Alerts pane included examples such as HTTP Only Site, Content Security Policy header not set, missing anti-clickjacking header, cookie SameSite observation and server information disclosure. The screenshot alone does not establish exploitability for each alert; inspect the saved ZAP report for full URL, evidence, confidence, risk and solution.

Finding 4 — Security-header/cookie configuration observations were reported by ZAP. Recommended action: review each alert, configure appropriate CSP and anti-clickjacking controls, and set session cookies with HttpOnly, Secure (when HTTPS is used) and a suitable SameSite value.

9. Findings Summary

Finding	Evidence	Recommended mitigation
SQL Injection	Crafted input returned multiple records.	Prepared statements; least privilege.
Reflected XSS	“XSS Test” alert appeared.	Context-aware output encoding.
Stored XSS	Stored guestbook payload triggered alert.	Encode output; sanitize allowed HTML.
ZAP configuration alerts	12 groups shown; scan console reported 87 alerts.	Review individual alerts and retest after fixes.

Risk labels for the real world must be assigned after considering impact, exposure, confidence and validation. The DVWA demonstrations are not evidence that an unrelated production website has the same weaknesses.

10. Remediation and Conclusion

Recommended remediation

• SQLi: use prepared statements, validate expected input types and restrict database privileges.

• XSS: contextually encode output, sanitize allowed HTML and review unsafe DOM APIs.

• Headers/cookies: apply suitable CSP and frame-ancestors protection; configure HttpOnly, Secure and SameSite attributes appropriately.

• ZAP: review the full exported report, validate each alert, fix confirmed issues and rerun the scan.

Conclusion

The Week 6 practical demonstrated the workflow of local web application testing: launching DVWA with Docker, observing SQL Injection and XSS behaviour, inspecting HTTP traffic in Burp Suite and completing an OWASP ZAP scan. The main learning is to record evidence accurately and treat scanner alerts as leads for validation, not automatic proof of exploitability.

References

• OWASP Top 10 — https://owasp.org/www-project-top-ten/

• DVWA — https://github.com/digininja/DVWA

• OWASP ZAP — https://www.zaproxy.org/

• PortSwigger Web Security Academy — https://portswigger.net/web-security

Prepared by Prajapati Meet Ghanshyam | DG Interns Hub | 28 October 2026